



Universidade do Minho
Escola de Engenharia

TP2

Redes de Computadores

Alexandre Gabriel Silva Machado (A106801)
~~Daniel Ramos de Almeida Marcos Costa (A108708)~~
Simão Pedro Ribeiro Santos (A109767)

04 de maio de 2026
Braga, Portugal

Índice de conteúdos

1. Introdução	1
1.1. Contexto e objetivos	1
1.2. Requisitos e âmbito do trabalho	1
1.3. Protocolos selecionados	2
2. Arquitetura e implementação	4
2.1. Visão global dos módulos da aplicação	4
2.2. Interface de execução e modos de funcionamento	5
2.3. Filtros e parâmetros disponíveis	6
2.4. Registo, persistência e estatísticas	8
3. Identificação e análise de protocolos	10
3.1. Desencapsulamento e parsing por camadas	10
3.2. Protocolos suportados e campos analisados	11
3.3. Rastreador de estado e deteção de interações características	13
3.3.1. Deteção de sessões TCP	15
3.3.2. Reconstrução virtual de Fragmentação	16
3.3.3. Identificação de padrões: ping, resolução ARP e traceroute	16
4. Demonstração e Validação	18
4.1. Ambiente emulado no CORE	18
4.2. Análise na interface real	19
4.3. Testes efetuados e resultados observados	19
4.4. Obstáculos encontrados e resolução	20
5. Considerações	22
5.1. Limitações da solução	22
5.2. Melhorias possíveis	22
5.2.1. Reconstrução de traceroute	23
6. Conclusão	24

Índice de figuras

Figura 1 – Ajuda da aplicação na linha de comandos	2
Figura 2 – Estrutura modular da aplicação	5
Figura 3 – Exemplo de execução da aplicação em modo live, evidenciando a identificação da interface de captura, a numeração sequencial dos pacotes e o formato do resumo apresentado na consola.	5
Figura 4 – Exemplo de execução da aplicação em modo offline, evidenciando a leitura de um ficheiro PCAP, a identificação da fonte e a consistência do formato de output relativamente ao modo live.	6
Figura 7 – Exemplo de detalhe textual de um pacote, com apresentação estruturada da informação por camadas, desde Ethernet até ao protocolo identificado no interior do datagrama IPv4.	11
Figura 8 – Exemplo de interação reconhecida pelo rastreador de estado, mostrando a referência entre pedido e resposta e a emissão do evento correspondente.	14
Figura 9 – Exemplo de deteção de uma sessão TCP, mostrando o reconhecimento simplificado do estabelecimento e da terminação da ligação através das flags observadas.	15
Figura 10 – Exemplo de reconstrução virtual de fragmentação IPv4, mostrando a associação entre fragmentos do mesmo datagrama e a deteção do conjunto completo.	16
Figura 11 – Exemplo de identificação heurística de possível traceroute, mostrando uma sequência de pedidos ICMP com valores de TTL crescentes e a emissão do evento correspondente.	17
Figura 12 – Exemplo de execução do sniffer em ambiente emulado no CORE, mostrando a deteção de pedidos e respostas ARP entre nós da mesma rede virtual.	18
Figura 13 – Exemplo de execução do sniffer numa interface real, mostrando a identificação de tráfego ICMP e a associação entre pedido e resposta no output da aplicação.	19

Índice de tabelas

Tabela 1 – Principais parâmetros de execução e filtragem suportados	7
Tabela 2 – Protocolos suportados pela aplicação e principais campos analisados.	13
Tabela 3 – Interações características detetadas pelo rastreador de estado.	14
Tabela 4 – Padrões de comunicação reconhecidos pela aplicação.	17
Tabela 5 – Síntese dos testes realizados e dos resultados observados	20

1. Introdução

1.1. Contexto e objetivos

O presente trabalho prático tem como objetivo o desenvolvimento de um *packet sniffer* capaz de capturar, identificar e classificar tráfego de rede, de modo semelhante a ferramentas como o *Wireshark*. Para além da captura em si, pretende-se extrair os campos mais relevantes dos cabeçalhos de cada pacote e interpretar o comportamento dos protocolos envolvidos, com o objetivo final de relacionar o tráfego observado com o funcionamento de cada protocolo e distinguir fases como o estabelecimento de ligação, a troca de mensagens e o término da comunicação.

Sabe-se que a comunicação em rede resulta da cooperação entre diferentes protocolos organizados em camadas, em que cada uma acrescenta as suas informações de controlo à totalidade da informação recebida da camada superior, encapsulando progressivamente a mensagem de modo a coordenar a sua entrega, encaminhamento, multiplexagem e interpretação no destino. No processo inverso, o desencapsulamento, cada camada remove o seu cabeçalho à medida que os dados sobem na pilha, até a mensagem original chegar ao seu destino.

No contexto deste *packet sniffer*, a análise de cada pacote capturado passa pela leitura ordenada desses cabeçalhos, permitindo observar diretamente o processo de encapsulamento e consolidar uma compreensão prática do funcionamento das comunicações em rede.

1.2. Requisitos e âmbito do trabalho

Quer-se satisfazer um conjunto de requisitos funcionais centrados na captura, observação e análise de tráfego de rede. De forma geral, a aplicação deve ser capaz de capturar pacotes numa interface de rede, apresentar informação relevante por pacote, suportar registo estruturado nos formatos TXT, CSV e JSON Lines, disponibilizar no final de cada sessão estatísticas por protocolo, principais emissores e eventos detetados, disponibilizar mecanismos de filtragem e suportar dois modos de execução – observação em tempo real e registo em ficheiro, com gravação opcional da captura bruta num ficheiro `.pcap`. Para além disso, deve conseguir reconhecer comportamentos típicos entre protocolos e quanto às trocas observadas.

Igualmente, pretende-se que a aplicação contemple duas fases de utilização distintas, sendo, numa primeira fase, validada num ambiente emulado no CORE (com cenários controlados para simular interações simples, como pedidos ARP ou trocas TCP)

e, numa segunda, executada sobre uma interface real do computador (com tráfego real e confirmando o reconhecimento correto dos protocolos fora da emulação).

Com este enquadramento, a aplicação foi desenvolvida em Python, com recurso à biblioteca Scapy, recomendada para este tipo de propósito – disponibiliza, numa única API de alto nível, suporte para captura, dissecação e escrita de pacotes, sem necessidade de lidar diretamente com *raw sockets*, o que simplifica significativamente a implementação e permite focar o desenvolvimento na lógica de análise protocolar.

O âmbito do projeto é condicionado por algumas restrições de utilização: a aplicação destina-se exclusivamente à inspeção passiva em redes autorizadas (como o ambiente CORE ou a interface do computador), uma vez que o trabalho consiste em observar e interpretar o tráfego, não interferir com ele. Não há qualquer componente de exploração ativa, injeção de tráfego ou recolha de dados sensíveis.

```
usage: main.py [-h] [-i INTERFACE] [-r PCAP] [-c COUNT] [--bpf BPF] [--ip IP] [--src-ip SRC_IP] [--dst-ip DST_IP] [--mac MAC] [--protocol PROTOCOL]
               [--src-port SRC_PORT] [--dst-port DST_PORT] [--fragmented] [--ip-id IP_ID] [--mf-only] [--write-pcap WRITE_PCAP] [--timeout TIMEOUT]
               [--log-file LOG_FILE] [--log-format {csv,json,txt}]

Packet sniffer em Python com Scapy.

options:
  -h, --help                show this help message and exit
  -i, --interface INTERFACE  interface de rede para captura em tempo real
  -r, --pcap PCAP            caminho para um ficheiro .pcap para leitura offline
  -c, --count COUNT          número máximo de pacotes a processar (0 significa sem limite)
  --bpf BPF                  expressão BPF bruta, por exemplo: 'tcp port 80'
  --ip IP                    filtrar por endereço IP
  --src-ip SRC_IP            filtrar por endereço IP de origem
  --dst-ip DST_IP            filtrar por endereço IP de destino
  --mac MAC                  filtrar por endereço MAC
  --protocol PROTOCOL        filtrar por protocolo: arp, ip, icmp, tcp ou udp
  --src-port SRC_PORT        filtrar por porta de origem TCP/UDP
  --dst-port DST_PORT        filtrar por porta de destino TCP/UDP
  --fragmented               aceitar apenas pacotes IPv4 fragmentados
  --ip-id IP_ID              filtrar por identificador IPv4
  --mf-only                  aceitar apenas pacotes IPv4 com a flag MF ativa
  --write-pcap WRITE_PCAP    ficheiro .pcap de saída para guardar a captura crua em modo live
  --timeout TIMEOUT          tempo limite da captura live, em segundos
  --log-file LOG_FILE        ficheiro de saída para logging estruturado
  --log-format {csv,json,txt} formato do log: txt, csv ou json
```

Figura 1 – Ajuda da aplicação na linha de comandos

1.3. Protocolos selecionados

Os protocolos analisados foram selecionados por serem representativos de diferentes camadas da pilha e suficientemente observáveis tanto em ambiente controlado como em interface real. A seleção abrange Ethernet, ARP, IPv4, ICMP, TCP e UDP, com identificação contextual adicional para DNS, DHCP e HTTP, cobrindo desde a camada de ligação até à camada de transporte com alguma visibilidade sobre serviços de aplicação comuns, sem *parsing* profundo de *payload* nem reconstrução de sessões.

Ao nível da camada de ligação, o Ethernet permite observar a comunicação ao nível da trama e dos endereços MAC. O ARP torna visível o processo de resolução entre endereços IP e endereços físicos, um mecanismo essencial da comunicação em rede local, normalmente invisível ao utilizador e indispensável na comunicação dentro de uma rede local e a troca *request/reply* constitui um exemplo simples, claro e fácil de desencadear. Na camada de rede, o IPv4 é a base do encaminhamento observado: o seu cabeçalho expõe endereços de origem e destino, TTL, protocolo encapsulado e campos de fragmentação (*identificador*, *fragment offset* e *flag MF*), o que permite identificar e associar fragmentos pertencentes ao mesmo datagrama. O ICMP cobre mensagens de

controle simples como `echo-request` e `echo-reply`. Na camada de transporte, o TCP permite analisar mecanismos com estado, nomeadamente o *three-way handshake* e a terminação por FIN ou RST, e o UDP representa o modelo sem ligação. Em conjunto, estes três protocolos contrastam abordagens distintas à comunicação entre processos.

Para os serviços de aplicação, optou-se por uma identificação baseada em portas: DNS na porta 53 (UDP), com reconhecimento de consultas e respostas, DHCP nas portas 67 e 68 (UDP), com identificação das mensagens `Discover`, `Offer`, `Request` e `ACK`, e HTTP na porta 80 (TCP). Esta abordagem aumenta a capacidade de relacionar o tráfego capturado com interações comuns sem transformar a aplicação num analisador profundo de protocolos de aplicação.

Note-se que a aplicação inclui ainda mecanismos complementares de interpretação (referências entre pedidos e respostas em ARP, ICMP e DNS, deteção heurística de *traceroute* por padrão de TTL crescente, e identificação de conjuntos de fragmentos IPv4 pertencentes ao mesmo datagrama) bem como uma consulta iterativa que permite inspecionar qualquer pacote procurando-o pelo seu número de sequência, indo assim além da identificação simples. Esses mecanismos são detalhados mais à frente.

A seleção cobre, assim, encapsulamento, resolução de endereços, mensagens de controle, transporte com e sem ligação, fragmentação e associação contextual a serviços de aplicação.

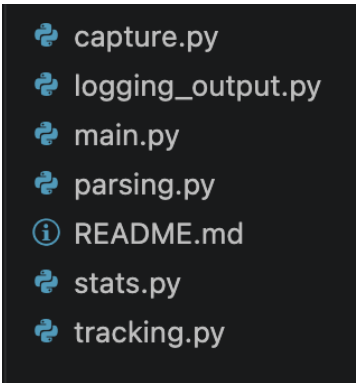
2. Arquitetura e implementação

2.1. Visão global dos módulos da aplicação

A aplicação foi organizada de forma modular, distribuindo o processamento por módulos especializados correspondentes às várias etapas do funcionamento: interpretação da linha de comandos, captura, análise protocolar, rastreo de estado, *logging* e estatísticas.

O ponto de entrada é `main.py`, responsável pela validação dos argumentos, preparação dos filtros, escolha do modo de funcionamento, apresentação do resumo final e ativação da consulta interativa de pacotes por número de sequência (tudo isto para além da definição da interface). A captura é tratada em `capture.py`, com contagem de pacotes, estado do rastreador, estatísticas acumuladas, histórico e escrita opcional para novo ficheiro `.pcap` e encaminhando cada pacote para uma rotina comum de processamento.

A análise protocolar está concentrada em `parsing.py`, responsável pela extração dos campos (de Ethernet, ARP, IPv4, etc., como explicado no subcapítulo anterior), produção dos resumos textuais, filtros simplificados em modo offline, formatação de detalhes para consulta interativa e identificação contextual de DNS, DHCP e HTTP. O rastreo de relações entre pacotes é tratado em `tracking.py`, que mantém estado mínimo em memória para associar pedidos e respostas ARP, ICMP e DNS, etapas do *handshake* e terminação TCP, padrões de possível *traceroute* (por deteção de padrões de TTL crescente) e agrupamento de fragmentos IPv4 pertencentes ao mesmo datagrama. A persistência é assegurada por `logging_output.py`, com escrita em TXT, CSV e JSON Lines, e as estatísticas finais são agregadas em `stats.py`.



- capture.py
- logging_output.py
- main.py
- parsing.py
- README.md
- stats.py
- tracking.py

2.2. Interface de execução e modos de funcionamento

A interação com a aplicação é feita inteiramente pela linha de comandos, centrada na escolha da fonte de captura, na definição opcional de filtros e na ativação dos mecanismos de registo e limitação da execução.

É aplicado o mesmo pipeline de análise, sendo a fonte identificada explicitamente no prefixo de cada linha do output. Em cada execução deve ser indicada exatamente uma fonte, e os argumentos são validados antes do início, incluindo protocolos suportados, formatos de endereços, coerência entre opções de *logging* e restrições como a disponibilidade de filtros BPF apenas em modo live.

A execução pode ser limitada por número de pacotes ou por tempo máximo de captura. No modo live, estes parâmetros permitem capturas curtas e controladas, no modo offline, a limitação por contagem permite trabalhar sobre uma parte do ficheiro `.pcap`.

Cada pacote é apresentado numa linha com número sequencial, identificação da fonte, *timestamp*, e resumo da informação relevante. Quando o rastreador associa pacotes entre si, o resumo inclui anotações como `request in line ...`, permitindo reconhecer relações entre pedidos e respostas diretamente no output.

No final, como já mencionado anteriormente, é apresentado um resumo da sessão com tipo e identificação da fonte, filtro configurado, total de pacotes processados, contagens por protocolo, principais emissores e eventos detetados. Havendo pelo menos um pacote processado, é disponibilizada a consulta que permite selecionar um pacote pelo número e visualizar o detalhe completo das camadas observadas.

```
alex@Alexs-MacBook-Air RCTP2PL68 % sudo python3 main.py -i en0 -c 4 --protocol icmp

[1] [live:en0] [20:37:38] Ethernet | IPv4 | ICMP | 172.26.42.238 -> 8.8.8.8 | ttl=64 | echo-request | 84 bytes
[2] [live:en0] [20:37:38] Ethernet | IPv4 | ICMP | 8.8.8.8 -> 172.26.42.238 | ttl=113 | echo-reply | 84 bytes | request in line 1
[event] ICMP reply recebido | 8.8.8.8 respondeu a 172.26.42.238
[3] [live:en0] [20:37:39] Ethernet | IPv4 | ICMP | 172.26.42.238 -> 8.8.8.8 | ttl=64 | echo-request | 84 bytes
[4] [live:en0] [20:37:39] Ethernet | IPv4 | ICMP | 8.8.8.8 -> 172.26.42.238 | ttl=113 | echo-reply | 84 bytes | request in line 3
[event] ICMP reply recebido | 8.8.8.8 respondeu a 172.26.42.238

Resumo:
tipo de fonte: live
fonte: en0
filtro configurado: --protocol icmp
pacotes processados: 4

Estatísticas:
protocolos:
  ICMP: 4 (100.0%)

top talkers:
  1. 172.26.42.238 - 2 pacotes
  2. 8.8.8.8 - 2 pacotes

eventos:
  ICMP reply recebido: 2

Selecione o pacote que quer analisar (ou prima 0 para terminar):
```

Figura 3 – Exemplo de execução da aplicação em modo live, evidenciando a identificação da interface de captura, a numeração sequencial dos pacotes e o formato do resumo apresentado na consola.

```
alex@Alexs-MacBook-Air RCTP2PL68 % python3 main.py -r captura.pcap -c 10

[1] [offline:captura.pcap] [20:33:31] Ethernet | IPv4 | TCP [ACK] | 18.97.36.52:443 -> 172.26.42.238:55994 | ttl=109 | 76 bytes
[2] [offline:captura.pcap] [20:33:31] Ethernet | IPv4 | TCP [ACK] | 172.26.42.238:55994 -> 18.97.36.52:443 | ttl=64 | 52 bytes
[3] [offline:captura.pcap] [20:33:31] Ethernet | IPv4 | TCP [ACK] | 172.26.42.238:55994 -> 18.97.36.52:443 | ttl=64 | 76 bytes
[4] [offline:captura.pcap] [20:33:31] Ethernet | IPv4 | TCP [ACK] | 172.26.42.238:55994 -> 18.97.36.52:443 | ttl=64 | 78 bytes
[5] [offline:captura.pcap] [20:33:31] Ethernet | IPv4 | TCP [ACK] | 18.97.36.52:443 -> 172.26.42.238:55994 | ttl=109 | 52 bytes
[6] [offline:captura.pcap] [20:33:31] Ethernet | IPv4 | TCP [ACK] | 18.97.36.52:443 -> 172.26.42.238:55994 | ttl=109 | 52 bytes
[7] [offline:captura.pcap] [20:33:31] Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 818 bytes
[8] [offline:captura.pcap] [20:33:31] Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 1497 bytes
[9] [offline:captura.pcap] [20:33:31] Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 1148 bytes
[10] [offline:captura.pcap] [20:33:31] Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 167 bytes

Resumo:
  tipo de fonte: offline
  fonte: captura.pcap
  filtro configurado: (sem filtro)
  pacotes processados: 10

Estatísticas:
  protocolos:
    TCP: 6 (60.0%)
    UDP: 4 (40.0%)

  top talkers:
    1. 172.26.42.238 - 6 pacotes
    2. 18.97.36.52 - 3 pacotes
    3. 194.210.238.205 - 1 pacote

  eventos:
    (sem eventos detetados)

Selecione o pacote que quer analisar (ou prima 0 para terminar):
```

Figura 4 – Exemplo de execução da aplicação em modo offline, evidenciando a leitura de um ficheiro PCAP, a identificação da fonte e a consistência do formato de output relativamente ao modo live.

2.3. Filtros e parâmetros disponíveis

Os parâmetros disponíveis agrupam-se em quatro categorias: seleção da fonte de captura, controlo da execução, filtros e mecanismos de persistência e *logging*. Deste modo, pode-se isolar o relevante e adequar a captura à observação que se pretenda.

No que respeita à filtragem, a aplicação suporta filtros simplificados por protocolo, endereço IP (origem, destino ou ambos), endereço MAC, porta (origem ou destino) e critérios relacionados com a fragmentação IPv4, nomeadamente, seleção exclusiva de pacotes fragmentados, filtragem por identificador IPv4 e filtragem pela *flag MF* ativa. Estes filtros são úteis em cenários de teste dirigidos onde se pretende isolar subconjuntos específicos de tráfego, por exemplo, um filtro por protocolo ICMP facilita a observação de mensagens *echo-request* e *echo-reply*, enquanto a combinação entre IP de origem, protocolo UDP e porta de destino permite restringir a análise a tráfego muito mais específico. No caso da fragmentação, os filtros disponíveis permitem ainda reduzir o ruído e apoiar a validação dessa funcionalidade. Em conjunto com os filtros BPF (disponíveis apenas em modo live), a filtragem funciona não só como conveniência operacional, mas também como mecanismo de apoio à demonstração e validação.

No caso do *logging*, é construído para cada pacote um registo estruturado que pode ser exportado em TXT (melhor legibilidade humana), CSV (ideal para análise tabular) ou JSON Lines (mais compatível com processamento automático), disponibilizando assim formas complementares de reutilização da informação capturada.

A Tabela 1 resume os principais parâmetros suportados pela aplicação, bem como a respetiva função no contexto da execução.

Categoria	Parâmetro	Função
Fonte de captura	<code>-i, --interface</code>	Seleciona a interface de rede a observar em modo live.
Fonte de captura	<code>-r, --pcap</code>	Lê pacotes a partir de um ficheiro .pcap em modo offline.
Controlo da execução	<code>-c, --count</code>	Limita o número máximo de pacotes processados.
Controlo da execução	<code>--timeout</code>	Define um tempo máximo de captura em modo live.
Filtros	<code>--protocol</code>	Filtra por protocolo principal suportado, como ARP, IP, ICMP, TCP ou UDP.
Filtros	<code>--ip</code>	Filtra por endereço IP, considerando origem ou destino.
Filtros	<code>--src-ip</code>	Filtra por endereço IP de origem.
Filtros	<code>--dst-ip</code>	Filtra por endereço IP de destino.
Filtros	<code>--mac</code>	Filtra por endereço MAC, considerando origem ou destino Ethernet.
Filtros	<code>--src-port</code>	Filtra por porta de origem em TCP ou UDP.
Filtros	<code>--dst-port</code>	Filtra por porta de destino em TCP ou UDP.
Filtros	<code>--fragmented</code>	Seleciona apenas pacotes IPv4 fragmentados.
Filtros	<code>--ip-id</code>	Filtra por identificador IPv4.
Filtros	<code>--mf-only</code>	Seleciona apenas pacotes com a flag More Fragments ativa.
Filtros	<code>--bpf</code>	Aplica uma expressão de filtragem BPF, disponível apenas em modo live.
Persistência e logging	<code>--write-pcap</code>	Guarda a captura original num novo ficheiro .pcap.
Persistência e logging	<code>--log-file</code>	Define o ficheiro de saída para logging estruturado.
Persistência e logging	<code>--log-format</code>	Seleciona o formato do log, entre TXT, CSV e JSON Lines.

Tabela 1 – Principais parâmetros de execução e filtragem suportados

2.4. Registo, persistência e estatísticas

A persistência em ficheiro `.pcap` permite conservar os pacotes de uma sessão ao vivo para reabertura posterior em modo offline ou inspeção com outras ferramentas compatíveis, sem depender novamente da geração do tráfego original.

Os registos estruturados produzidos durante o processamento assentam num princípio de consistência: para cada pacote, é construído um registo com os campos mais relevantes – metadados da execução, identificação da fonte, protocolo principal, endereços, portas quando aplicável, informação de fragmentação e resumo textual – que pode depois ser exportado nos formatos já descritos (TXT, CSV e JSON Lines). Desta forma, o registo constitui uma extensão estruturada da mesma informação produzida durante o processamento, e não uma via paralela desligada da consola. Está representado na Figura 6.

A componente estatística, apresentada no final de cada execução, inclui o tipo e identificação da fonte, o filtro configurado e o total de pacotes processados. A este nível juntam-se estatísticas agregadas por protocolo, que permitem perceber rapidamente a composição geral da captura, distinguindo, por exemplo, quantos pacotes foram classificados como ARP, ICMP, TCP, UDP ou outro tipo não suportado. A identificação dos principais emissores acrescenta uma leitura quantitativa sobre os endereços IP de origem mais ativos durante a execução. Por fim, a contagem de eventos detetados resume o resultado do rastreador de estado: quantas respostas ICMP foram reconhecidas, quantas resoluções ARP ficaram completas, quantos *handshakes* TCP foram identificados ou quantos conjuntos de fragmentos IPv4 foram associados logicamente. Estas estatísticas complementam a análise detalhada dos pacotes com uma perspetiva global e imediatamente interpretável. Estão representadas na Figura 5.

```
alex@Alexs-MacBook-Air RCTP2PL68 % python3 main.py -r demo.pcap

[1] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | ICMP | 172.26.42.238 -> 8.8.8.8 | ttl=64 | echo-request | 84 bytes
[2] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | ICMP | 8.8.8.8 -> 172.26.42.238 | ttl=113 | echo-reply | 84 bytes | request in line 1
[eventos] ICMP reply recebido | 8.8.8.8 respondeu a 172.26.42.238
[3] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 818 bytes
[4] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 1497 bytes
[5] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 1151 bytes
[6] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 167 bytes
[7] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 71 bytes
[8] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1223 bytes
[9] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1228 bytes
[10] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 63 bytes
[11] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1228 bytes
[12] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1228 bytes
[13] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1228 bytes
[14] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1228 bytes
[15] [offline:demo.pcap] [20:41:34] Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1228 bytes

Resumo:
tipo de fonte: offline
fonte: demo.pcap
filtro configurado: (sem filtro)
pacotes processados: 15

Estatísticas:
protocolos:
ICMP: 2 (13.3%)
UDP: 13 (86.7%)

top talkers:
1. 194.210.238.205 - 8 pacotes
2. 172.26.42.238 - 6 pacotes
3. 8.8.8.8 - 1 pacote

eventos:
ICMP reply recebido: 1

Selecione o pacote que quer analisar (ou prima 0 para terminar):
```

Figura 5 – Exemplo do resumo final da execução.

```
1 packet_number,timestamp,source_type,source_name,protocol,src_ip,dst_ip,src_port,dst_port,ttl,length,ip_id,fragment_offset,more_fragments,service,summary
2 1,2026-05-04T20:41:34,offline,demo.pcap,ICMP,172.26.42.238,8.8.8.8,,64,84,27402,0,False,,Ethernet | IPv4 | ICMP | 172.26.42.238 -> 8.8.8.8 | ttl=64 | echo-request | 84 bytes
3 2,2026-05-04T20:41:34,offline,demo.pcap,ICMP,8.8.8.8,172.26.42.238,,113,84,0,0,False,,Ethernet | IPv4 | ICMP | 8.8.8.8 -> 172.26.42.238 | ttl=113 | echo-reply | 84 bytes | request in line 1
4 3,2026-05-04T20:41:34,offline,demo.pcap,UDP,172.26.42.238,194.210.238.205,63179,443,64,818,0,0,False,,Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 818 bytes
5 4,2026-05-04T20:41:34,offline,demo.pcap,UDP,172.26.42.238,194.210.238.205,63179,443,64,1497,0,0,False,,Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 1497 bytes
6 5,2026-05-04T20:41:34,offline,demo.pcap,UDP,172.26.42.238,194.210.238.205,63179,443,64,1151,0,0,False,,Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 1151 bytes
7 6,2026-05-04T20:41:34,offline,demo.pcap,UDP,194.210.238.205,172.26.42.238,443,63179,119,167,8842,0,False,,Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 167 bytes
8 7,2026-05-04T20:41:34,offline,demo.pcap,UDP,172.26.42.238,194.210.238.205,63179,443,64,71,0,0,False,,Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 71 bytes
9 8,2026-05-04T20:41:34,offline,demo.pcap,UDP,194.210.238.205,172.26.42.238,443,63179,119,1223,8844,0,False,,Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1223 bytes
10 9,2026-05-04T20:41:34,offline,demo.pcap,UDP,194.210.238.205,172.26.42.238,443,63179,119,1228,8845,0,False,,Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1228 bytes
11 10,2026-05-04T20:41:34,offline,demo.pcap,UDP,172.26.42.238,194.210.238.205,63179,443,64,63,0,0,False,,Ethernet | IPv4 | UDP | 172.26.42.238:63179 -> 194.210.238.205:443 | ttl=64 | 63 bytes
12 11,2026-05-04T20:41:34,offline,demo.pcap,UDP,194.210.238.205,172.26.42.238,443,63179,119,1228,8846,0,False,,Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1228 bytes
13 12,2026-05-04T20:41:34,offline,demo.pcap,UDP,194.210.238.205,172.26.42.238,443,63179,119,1228,8847,0,False,,Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1228 bytes
14 13,2026-05-04T20:41:34,offline,demo.pcap,UDP,194.210.238.205,172.26.42.238,443,63179,119,1228,8848,0,False,,Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1228 bytes
15 14,2026-05-04T20:41:34,offline,demo.pcap,UDP,194.210.238.205,172.26.42.238,443,63179,119,1228,8849,0,False,,Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1228 bytes
16 15,2026-05-04T20:41:34,offline,demo.pcap,UDP,194.210.238.205,172.26.42.238,443,63179,119,1228,8850,0,False,,Ethernet | IPv4 | UDP | 194.210.238.205:443 -> 172.26.42.238:63179 | ttl=119 | 1228 bytes
```

Figura 6 – Exemplo de registo persistido pela aplicação em formato CSV, ilustrando a exportação estruturada da informação capturada para análise posterior.

3. Identificação e análise de protocolos

3.1. Desencapsulamento e parsing por camadas

A análise de cada pacote segue uma leitura progressiva por camadas, orientada pelos campos de identificação presentes em cada cabeçalho, conforme explicado logo na introdução.

O primeiro nível de observação corresponde à trama Ethernet: o valor do campo `EtherType` determina o passo seguinte: se corresponde a ARP, o pacote é interpretado diretamente nesse âmbito, se corresponde a IPv4, a análise prossegue para o cabeçalho IP, caso contrário, a aplicação assinala explicitamente essa situação no *output*, mantendo ainda assim a identificação da camada Ethernet e do tipo encapsulado.

Quando o pacote contém IPv4, são extraídos os campos relevantes do cabeçalho, incluindo endereços de origem e destino, TTL, comprimento total e protocolo encapsulado, bem como campos de fragmentação quando aplicável. O campo `Protocol` do cabeçalho IPv4 determina então a transição para a camada de transporte, distinguindo entre ICMP, TCP, UDP ou valor não suportado. Ao nível da camada de transporte, o parsing segue caminhos distintos: no ICMP, observam-se tipo e código, reconhecendo padrões como `echo-request`, `echo-reply` e mensagens relevantes para *traceroute*, no TCP, analisam-se as *flags* de estado da comunicação, no UDP, extraem-se as portas, possibilitando a identificação contextual de serviços por porta. Em todos os casos, o objetivo é extrair os campos suficientes para caracterizar o comportamento típico dos protocolos, sem reconstruir sessões completas nem interpretar exaustivamente o *payload*.

Este processo de desencapsulamento é refletido diretamente no output: o resumo de cada pacote apresenta a sequência lógica das camadas reconhecidas, de Ethernet até ao protocolo de transporte. Elementos de identificação contextual como DNS, DHCP ou HTTP surgem como enriquecimento conservador do resumo, sem alterar a lógica principal do *parsing* por camadas. Desta forma, cada decisão de *parsing* pode ser associada a um campo concreto do cabeçalho em análise, tornando o funcionamento da aplicação diretamente explicável a partir da organização protocolar do tráfego observado.

```

[1] [live:en0] [15:30:37] Ethernet | IPv4 | 172.26.204.185 -> 8.8.8.8 | ttl=64 | ICMP | echo-request | 84 bytes
[2] [live:en0] [15:30:37] Ethernet | IPv4 | 8.8.8.8 -> 172.26.204.185 | ttl=113 | ICMP | echo-reply | 84 bytes | request in line 1
[event] ICMP reply recebido | 8.8.8.8 respondeu a 172.26.204.185
[3] [live:en0] [15:30:38] Ethernet | IPv4 | 172.26.204.185 -> 8.8.8.8 | ttl=64 | ICMP | echo-request | 84 bytes
[4] [live:en0] [15:30:38] Ethernet | IPv4 | 8.8.8.8 -> 172.26.204.185 | ttl=113 | ICMP | echo-reply | 84 bytes | request in line 3
[event] ICMP reply recebido | 8.8.8.8 respondeu a 172.26.204.185
[5] [live:en0] [15:30:39] Ethernet | IPv4 | 172.26.204.185 -> 8.8.8.8 | ttl=64 | ICMP | echo-request | 84 bytes
[6] [live:en0] [15:30:39] Ethernet | IPv4 | 8.8.8.8 -> 172.26.204.185 | ttl=113 | ICMP | echo-reply | 84 bytes | request in line 5
[event] ICMP reply recebido | 8.8.8.8 respondeu a 172.26.204.185

Resumo:
tipo de fonte: live
fonte: en0
filtro configurado: --protocol icmp
pacotes processados: 6

Estatísticas:
protocolos:
  ICMP: 6 (100.0%)

top talkers:
  1. 172.26.204.185 - 3 pacotes
  2. 8.8.8.8 - 3 pacotes

eventos:
  ICMP reply recebido: 3

[Selecione o pacote que quer analisar (ou prima 0 para terminar): 2

Detalhe do pacote 2
tipo de fonte: live
fonte: en0
timestamp: 2026-04-28T15:30:37
resumo: Ethernet | IPv4 | 8.8.8.8 -> 172.26.204.185 | ttl=113 | ICMP | echo-reply | 84 bytes | request in line 1

Ethernet:
src_mac: d4:7f:35:16:e2:9f
dst_mac: e2:1d:01:65:f6:ea
ethertype: IPv4 (0x0800)

IPv4:
src_ip: 8.8.8.8
dst_ip: 172.26.204.185
ttl: 113
length: 84
protocol: ICMP
ip_id: 0
fragment_offset: 0
more_fragments: False

ICMP:
type: 0
code: 0
name: echo-reply

```

Figura 7 – Exemplo de detalhe textual de um pacote, com apresentação estruturada da informação por camadas, desde Ethernet até ao protocolo identificado no interior do datagrama IPv4.

3.2. Protocolos suportados e campos analisados

Ao nível da camada Ethernet, a aplicação observa os endereços MAC de origem e de destino e o valor do campo `EtherType`. Quando o valor observado não corresponde a um protocolo suportado, essa situação é assinalada no output, preservando, ainda assim, a identificação da camada Ethernet e do tipo encapsulado.

No caso do ARP, a aplicação analisa o tipo de operação, os endereços IP envolvidos e os endereços MAC associados à resolução em curso – informação suficiente para distinguir pedidos e respostas e para relacionar pares *request/reply* quando o rastreador encontra correspondência. Este protocolo é particularmente útil porque torna visível o mecanismo de resolução entre endereços lógicos e físicos, normalmente oculto ao utilizador mas indispensável à comunicação em redes locais.

Para o IPv4, são extraídos endereços de origem e destino, TTL, comprimento total, o campo `Protocol` e, quando relevante, os campos de fragmentação (*identificador*, *fragment offset* e *flag MF*), que apoiam a deteção e associação lógica de fragmentos pertencentes ao mesmo datagrama.

A partir do campo `Protocol`, a análise prossegue por caminho distinto conforme o protocolo encapsulado. Para ICMP, observam-se tipo e código, permitindo distinguir `echo-request`, `echo-reply` e mensagens de controlo associadas a TTL excedido. Para TCP e UDP, extraem-se as portas de origem e destino, no caso do TCP, são ainda observadas as *flags* `SYN`, `ACK`, `FIN` e `RST`, essenciais para a interpretação do estado da comunicação.

As portas observadas em TCP e UDP permitem ainda associações contextuais conservadoras a serviços de aplicação: a porta UDP 53 indica DNS, com distinção entre consultas e respostas quando a estrutura do pacote o permite de forma clara, as portas UDP 67 e 68 indicam DHCP, com reconhecimento das mensagens `Discover`, `Offer`, `Request` e `ACK`, e a porta TCP 80 indica HTTP. Estas associações não substituem uma análise profunda dos protocolos de aplicação, mas enriquecem o output e facilitam a leitura funcional do tráfego observado.

Protocolo	Camada	Como é identificado	Campos analisados
Ethernet	Ligação (L2)	Presença da trama Ethernet na captura	<code>src_mac</code> , <code>dst_mac</code> , <code>EtherType</code>
ARP	Ligação (L2)	<code>EtherType</code> correspondente a ARP	operação, IP de origem/destino, MAC de origem/destino
IPv4	Rede (L3)	<code>EtherType</code> correspondente a IPv4	<code>src_ip</code> , <code>dst_ip</code> , <code>ttl</code> , <code>length</code> , <code>Protocol</code> , <code>ip_id</code> , <code>fragment_offset</code> , <code>more_fragments</code>
ICMP	Encapsulado em IPv4 (L3)	campo <code>Protocol</code> do IPv4 com valor correspondente a ICMP	<code>type</code> , <code>code</code> , nome resumido da mensagem
TCP	Encapsulado em IPv4 (L4)	campo <code>Protocol</code> do IPv4 com valor correspondente a TCP	<code>src_port</code> , <code>dst_port</code> , flags TCP
UDP	Encapsulado em IPv4 (L4)	campo <code>Protocol</code> do IPv4 com valor correspondente a UDP	<code>src_port</code> , <code>dst_port</code>
DNS	Identificação contextual (L7)	associação conservadora à porta 53 e, quando possível, presença de estrutura DNS	consulta/resposta, serviço DNS
DHCP	Identificação contextual (L7)	associação conservadora às portas UDP 67/68 e, quando possível, opções DHCP	tipo de mensagem DHCP

HTTP	Identificação contextual (L7)	associação conservadora à porta TCP 80	identificação contextual do serviço
------	-------------------------------	--	-------------------------------------

Tabela 2 – Protocolos suportados pela aplicação e principais campos analisados.

Os campos selecionados para análise atingem equilíbrio entre relevância e simplicidade: em vez de tentar cobrir exaustivamente todos os campos de cada cabeçalho, optou-se por aqueles que permitem justificar a identificação protocolar, caracterizar o papel do pacote na comunicação e relacionar o output produzido com os conceitos estudados na unidade curricular. Assim, o *sniffer* não apenas indica que protocolo está presente, mas mostra de forma objetiva que elementos do cabeçalho sustentam essa conclusão.

3.3. Rastreador de estado e detecção de interações características

O rastreador de estado é um mecanismo leve, em memória, baseado na observação incremental dos pacotes à medida que são processados. Sempre que um novo pacote é recebido, a aplicação verifica se corresponde a algum padrão já em acompanhamento ou se deve dar origem a um novo estado interno.

A forma mais visível de atuação do rastreador é a criação de referências entre pacotes relacionados: quando um pacote constitui resposta a um pedido anteriormente observado, o resumo textual é enriquecido com anotações como `request in line ...`, permitindo perceber imediatamente a correspondência entre os dois momentos da interação.

Para além dessas referências, o rastreador gera também eventos textuais independentes quando certos padrões ficam suficientemente confirmados: em ARP, assinala a resolução de um endereço IP para um endereço MAC, em ICMP, sinaliza a receção de um `echo-reply` associado ao `echo-request` anterior, em TCP, acompanha as *flags* relevantes para identificar a conclusão do *three-way handshake* ou a terminação por FIN ou RST, em *traceroute*, deteta um padrão heurístico de TTL crescente e emite uma indicação quando a sequência é suficientemente consistente, na fragmentação IPv4, agrupa fragmentos por identificador e emite um evento quando o conjunto parece cobrir o datagrama completo.

Esta componente foi implementada com um critério propositadamente conservador: não pretende igualar motores completos de reconstrução de fluxos nem garantir exatidão absoluta em todos os cenários. Os mecanismos adotados dependem da ordem de chegada dos pacotes, da visibilidade efetiva do tráfego e da simplicidade do padrão em causa, de forma suficiente para demonstrar, de forma prática, como a análise de tráfego beneficia da manutenção de estado entre pacotes sucessivos. Não substitui a análise por campos individuais.

Interação	Critério geral de detecção	Resultado visível no output
ARP request/reply	associação entre pedido ARP e resposta correspondente	request in line ... e evento ARP resolvido
ICMP echo-request/echo-reply	associação entre pedido ICMP e resposta correspondente	request in line ... e evento ICMP reply recebido
DNS query/response	associação entre query DNS e resposta correspondente	request in line ... no resumo da resposta
TCP handshake	observação sequencial simplificada de SYN, SYN-ACK e ACK	evento TCP handshake concluído
TCP terminação	observação de FIN ou RST em fluxo acompanhado	evento TCP sessão terminada
Possível traceroute	padrão heurístico de TTL crescente	evento Possível traceroute detetado
Fragmentação IPv4	agrupamento de fragmentos do mesmo datagrama	referências entre fragmentos e evento Fragmentos IPv4 completos

Tabela 3 – Interações características detetadas pelo rastreador de estado.

```
[1] [live:en0] [10:28:36] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=64 | echo-request | 84 bytes
[2] [live:en0] [10:28:36] Ethernet | IPv4 | ICMP | 8.8.8.8 -> 192.168.1.72 | ttl=116 | echo-reply | 84 bytes | request in line 1
[evento] ICMP reply recebido | 8.8.8.8 respondeu a 192.168.1.72
[3] [live:en0] [10:28:37] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=64 | echo-request | 84 bytes
[4] [live:en0] [10:28:37] Ethernet | IPv4 | ICMP | 8.8.8.8 -> 192.168.1.72 | ttl=116 | echo-reply | 84 bytes | request in line 3
[evento] ICMP reply recebido | 8.8.8.8 respondeu a 192.168.1.72
[5] [live:en0] [10:28:38] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=64 | echo-request | 84 bytes
[6] [live:en0] [10:28:38] Ethernet | IPv4 | ICMP | 8.8.8.8 -> 192.168.1.72 | ttl=116 | echo-reply | 84 bytes | request in line 5
[evento] ICMP reply recebido | 8.8.8.8 respondeu a 192.168.1.72
[7] [live:en0] [10:28:39] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=64 | echo-request | 84 bytes
[8] [live:en0] [10:28:39] Ethernet | IPv4 | ICMP | 8.8.8.8 -> 192.168.1.72 | ttl=116 | echo-reply | 84 bytes | request in line 7
[evento] ICMP reply recebido | 8.8.8.8 respondeu a 192.168.1.72
[9] [live:en0] [10:28:40] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=64 | echo-request | 84 bytes
[10] [live:en0] [10:28:40] Ethernet | IPv4 | ICMP | 8.8.8.8 -> 192.168.1.72 | ttl=116 | echo-reply | 84 bytes | request in line 9
[evento] ICMP reply recebido | 8.8.8.8 respondeu a 192.168.1.72
^C
Resumo:
  tipo de fonte: live
  fonte: en0
  filtro configurado: --protocol icmp
  pacotes processados: 10

Estatísticas:
  protocolos:
    ICMP: 10 (100.0%)

  top talkers:
    1. 192.168.1.72 - 5 pacotes
    2. 8.8.8.8 - 5 pacotes

  eventos:
    ICMP reply recebido: 5
```

Figura 8 – Exemplo de interação reconhecida pelo rastreador de estado, mostrando a referência entre pedido e resposta e a emissão do evento correspondente.

Detalham-se nos subcapítulos seguintes (3.3.1., 3.3.2., 3.3.3.) a detecção de sessões TCP, a reconstrução virtual de fragmentação e a identificação de padrões (de seu exemplo o próprio *traceroute*).

3.3.1. Detecção de sessões TCP

A detecção do estabelecimento de ligação assenta numa versão simplificada do *three-way handshake*. Quando a aplicação observa um segmento com a flag **SYN**, regista internamente esse início de fluxo, se surgir posteriormente um **SYN-ACK** no sentido inverso, o estado é atualizado e finalmente, quando é observado um **ACK** coerente com essa sequência, a aplicação considera o *handshake* concluído e emite o evento **TCP handshake concluído**. Para além do estabelecimento, a aplicação observa também a presença de flags **FIN** ou **RST** em fluxos já acompanhados, emitindo nesse caso o evento **TCP sessão terminada** com indicação do tipo de terminação observado.

A detecção é feita com base numa chave composta pelos quatro extremos lógicos da comunicação (IP de origem, porta de origem, IP de destino e porta de destino) o que permite distinguir fluxos diferentes mesmo quando coexistem vários segmentos entre os mesmos *hosts*.

Como já frisado, este mecanismo faz o melhor que consegue com a informação que tem disponível: se algum pacote intermédio não for observado, ou se a captura começar a meio da sessão, o rastreador pode não reconhecer corretamente o *handshake* ou a terminação.

```
[1] [live:en0] [10:33:54] Ethernet | IPv4 | TCP [SYN] | 192.168.1.72:56348 -> 104.20.23.154:80 | ttl=64 | HTTP | 64 bytes
[2] [live:en0] [10:33:54] Ethernet | IPv4 | TCP [SYN-ACK] | 104.20.23.154:80 -> 192.168.1.72:56348 | ttl=58 | HTTP | 60 bytes
[3] [live:en0] [10:33:54] Ethernet | IPv4 | TCP [ACK] | 192.168.1.72:56348 -> 104.20.23.154:80 | ttl=64 | HTTP | 52 bytes
[evento] TCP handshake concluído | 192.168.1.72:56348 -> 104.20.23.154:80
[4] [live:en0] [10:33:54] Ethernet | IPv4 | TCP [ACK] | 192.168.1.72:56348 -> 104.20.23.154:80 | ttl=64 | HTTP | 126 bytes
[5] [live:en0] [10:33:54] Ethernet | IPv4 | TCP [ACK] | 104.20.23.154:80 -> 192.168.1.72:56348 | ttl=58 | HTTP | 52 bytes
[6] [live:en0] [10:33:54] Ethernet | IPv4 | TCP [ACK] | 104.20.23.154:80 -> 192.168.1.72:56348 | ttl=58 | HTTP | 890 bytes
[7] [live:en0] [10:33:54] Ethernet | IPv4 | TCP [ACK] | 192.168.1.72:56348 -> 104.20.23.154:80 | ttl=64 | HTTP | 52 bytes
[8] [live:en0] [10:33:54] Ethernet | IPv4 | TCP [ACK] | 104.20.23.154:80 -> 192.168.1.72:56348 | ttl=58 | HTTP | 57 bytes
[9] [live:en0] [10:33:54] Ethernet | IPv4 | TCP [ACK] | 192.168.1.72:56348 -> 104.20.23.154:80 | ttl=64 | HTTP | 52 bytes
[10] [live:en0] [10:33:54] Ethernet | IPv4 | TCP [FIN-ACK] | 192.168.1.72:56348 -> 104.20.23.154:80 | ttl=64 | HTTP | 52 bytes
[evento] TCP sessão terminada | 192.168.1.72:56348 -> 104.20.23.154:80 | FIN
[11] [live:en0] [10:33:54] Ethernet | IPv4 | TCP [FIN-ACK] | 104.20.23.154:80 -> 192.168.1.72:56348 | ttl=58 | HTTP | 52 bytes
[evento] TCP sessão terminada | 104.20.23.154:80 -> 192.168.1.72:56348 | FIN
[12] [live:en0] [10:33:54] Ethernet | IPv4 | TCP [ACK] | 192.168.1.72:56348 -> 104.20.23.154:80 | ttl=64 | HTTP | 52 bytes
^C
Resumo:
tipo de fonte: live
fonte: en0
filtro configurado: --protocol tcp
pacotes processados: 12

Estatísticas:
protocolos:
  TCP: 12 (100.0%)

top talkers:
  1. 192.168.1.72 - 7 pacotes
  2. 104.20.23.154 - 5 pacotes

eventos:
  TCP handshake concluído: 1
  TCP sessão terminada: 2
```

Figura 9 – Exemplo de detecção de uma sessão TCP, mostrando o reconhecimento simplificado do estabelecimento e da terminação da ligação através das flags observadas.

3.3.2. Reconstrução virtual de Fragmentação

O agrupamento dos fragmentos é feito com base numa chave composta pelos endereços IP de origem e destino, pelo identificador IPv4 e pelo protocolo encapsulado. Sempre que um pacote IPv4 apresenta *fragment offset* diferente de zero ou a *flag MF* ativa, a aplicação considera-o parte de um conjunto fragmentado e passa a acompanhá-lo internamente, registando o intervalo de bytes correspondente e a linha em que surgiu no output.

Este acompanhamento reflete-se diretamente no output: o primeiro fragmento surge com os campos normais de fragmentação (identificador, *offset* e *flag MF*), enquanto os fragmentos seguintes incluem uma referência textual ao conjunto já observado (por exemplo, *fragmento do conjunto em 211 e 212*) permitindo perceber visualmente que os pacotes observados são partes sucessivas do mesmo datagrama.

Quando é observado um fragmento sem a *flag MF*, a aplicação passa a conhecer o comprimento total esperado do datagrama e verifica se a união dos intervalos de bytes já registados cobre toda essa extensão. Se isso acontecer, é emitido o evento *Fragmentos IPv4 completos*. Importa sublinhar que esta reconstrução é apenas virtual: a aplicação não recompõe o conteúdo binário original, limitando-se a detetar, agrupar e relacionar fragmentos com base nos campos do cabeçalho, o que é suficiente para apoiar a demonstração da fragmentação no contexto deste trabalho.

```
[1] [offline:fragmentado.pcap] [10:42:30] IPv4 | UDP | 10.0.0.1:1234 -> 10.0.0.2:9999 | ttl=64 | id=12345 | offset=0 | MF | 1420 bytes
[2] [offline:fragmentado.pcap] [10:42:30] IPv4 | UDP | 10.0.0.1 -> 10.0.0.2 | ttl=64 | id=12345 | offset=1400 | MF | fragmento do conjunto em 1 | 1420 bytes
[3] [offline:fragmentado.pcap] [10:42:30] IPv4 | UDP | 10.0.0.1 -> 10.0.0.2 | ttl=64 | id=12345 | offset=2800 | fragmento do conjunto em 1 e 2 | 1228 bytes
[evento] Fragmentos IPv4 completos | 10.0.0.1 -> 10.0.0.2 | id=12345

Resumo:
  tipo de fonte: offline
  fonte: fragmentado.pcap
  filtro configurado: (sem filtro)
  pacotes processados: 3

Estatísticas:
  protocolos:
    UDP: 3 (100.0%)

top talkers:
  1. 10.0.0.1 - 3 pacotes

eventos:
  Fragmentos IPv4 completos: 1
```

Figura 10 – Exemplo de reconstrução virtual de fragmentação IPv4, mostrando a associação entre fragmentos do mesmo datagrama e a deteção do conjunto completo.

3.3.3. Identificação de padrões: ping, resolução ARP e traceroute

No caso do *ping*, a aplicação reconhece o padrão ICMP associado através da observação de mensagens *echo-request* e *echo-reply*. Quando é detetado um pedido deste tipo, a informação necessária é registada internamente para o relacionar com uma resposta posterior, se for observado o *echo-reply* correspondente, o resumo inclui uma referência à linha do pedido original e é emitido o evento *ICMP reply recebido*.

Na resolução ARP, o princípio é semelhante: a aplicação acompanha pedidos ARP e tenta associá-los à resposta no sentido inverso. Quando essa correspondência é

encontrada, o resumo da resposta inclui a referência à linha do pedido original e é emitido o evento ARP resolvido, indicando a associação observada entre endereço IP e endereço MAC.

A identificação de possível *traceroute* é com base na observação de um padrão de TTL crescente em pacotes pertencentes ao mesmo fluxo lógico. Esta abordagem pode não cobrir todos os casos possíveis com exatidão absoluta, mas é suficiente para reconhecer o padrão mais característico desta ferramenta em contexto de teste e demonstração.

```
[1] [live:en0] [10:55:00] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=1 | echo-request | 48 bytes
[2] [live:en0] [10:55:00] Ethernet | IPv4 | ICMP | 192.168.1.1 -> 192.168.1.72 | ttl=64 | type=11 | code=0 | 76 bytes
[3] [live:en0] [10:55:00] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=1 | echo-request | 48 bytes
[4] [live:en0] [10:55:00] Ethernet | IPv4 | ICMP | 192.168.1.1 -> 192.168.1.72 | ttl=64 | type=11 | code=0 | 76 bytes
[5] [live:en0] [10:55:00] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=1 | echo-request | 48 bytes
[6] [live:en0] [10:55:00] Ethernet | IPv4 | ICMP | 192.168.1.1 -> 192.168.1.72 | ttl=64 | type=11 | code=0 | 76 bytes
[7] [live:en0] [10:55:00] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=2 | echo-request | 48 bytes
[8] [live:en0] [10:55:05] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=2 | echo-request | 48 bytes
[9] [live:en0] [10:55:10] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=2 | echo-request | 48 bytes
[10] [live:en0] [10:55:15] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=3 | echo-request | 48 bytes
[11] [live:en0] [10:55:15] Ethernet | IPv4 | ICMP | 10.137.216.81 -> 192.168.1.72 | ttl=253 | type=11 | code=0 | 96 bytes
[12] [live:en0] [10:55:15] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=3 | echo-request | 48 bytes
[13] [live:en0] [10:55:15] Ethernet | IPv4 | ICMP | 10.137.216.81 -> 192.168.1.72 | ttl=253 | type=11 | code=0 | 96 bytes
[14] [live:en0] [10:55:15] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=3 | echo-request | 48 bytes
[15] [live:en0] [10:55:15] Ethernet | IPv4 | ICMP | 10.137.216.81 -> 192.168.1.72 | ttl=253 | type=11 | code=0 | 96 bytes
[16] [live:en0] [10:55:15] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=4 | echo-request | 48 bytes
[evento] Possível traceroute detetado | 192.168.1.72 -> 8.8.8.8
```

Figura 11 – Exemplo de identificação heurística de possível traceroute, mostrando uma sequência de pedidos ICMP com valores de TTL crescentes e a emissão do evento correspondente.

Padrão	Base de identificação	Resultado visível no output
Ping	associação entre <code>echo-request</code> e <code>echo-reply</code> em ICMP	<code>request in line ...</code> e evento <code>ICMP reply recebido</code>
Resolução ARP	associação entre pedido ARP e resposta correspondente	<code>request in line ...</code> e evento <code>ARP resolvido</code>
Possível traceroute	observação heurística de TTL crescente num mesmo fluxo lógico	evento <code>Possível traceroute detetado</code>

Tabela 4 – Padrões de comunicação reconhecidos pela aplicação.

4. Demonstração e Validação

4.1. Ambiente emulado no CORE

A validação em ambiente emulado foi realizada com recurso ao CORE, que permite gerar tráfego dirigido entre nós virtuais em condições mais previsíveis do que numa interface real, reduzindo ruído protocolar irrelevante e facilitando a associação entre os pacotes capturados e as interações efetivamente provocadas.

O projeto foi executado a partir de um nó do CORE com acesso ao diretório /home/core/Desktop/RCTP2PL68. Para permitir a utilização da biblioteca Scapy no interior do nó, foi necessário definir explicitamente a variável `PYTHONPATH` com o caminho correspondente à instalação disponível no sistema anfitrião virtualizado. A execução foi feita diretamente no nó, sobre a interface virtual `eth0`.

Assim, foi possível observar interações ARP entre dois nós ligados na mesma rede virtual. O sniffer executado em `n1` capturou corretamente pedidos e respostas ARP, incluindo a referência `request in line ...` e o evento `ARP resolvido`, demonstrando que a aplicação consegue não só identificar pacotes ARP individualmente, mas também reconhecer a relação lógica entre pedido e resposta num cenário controlado.

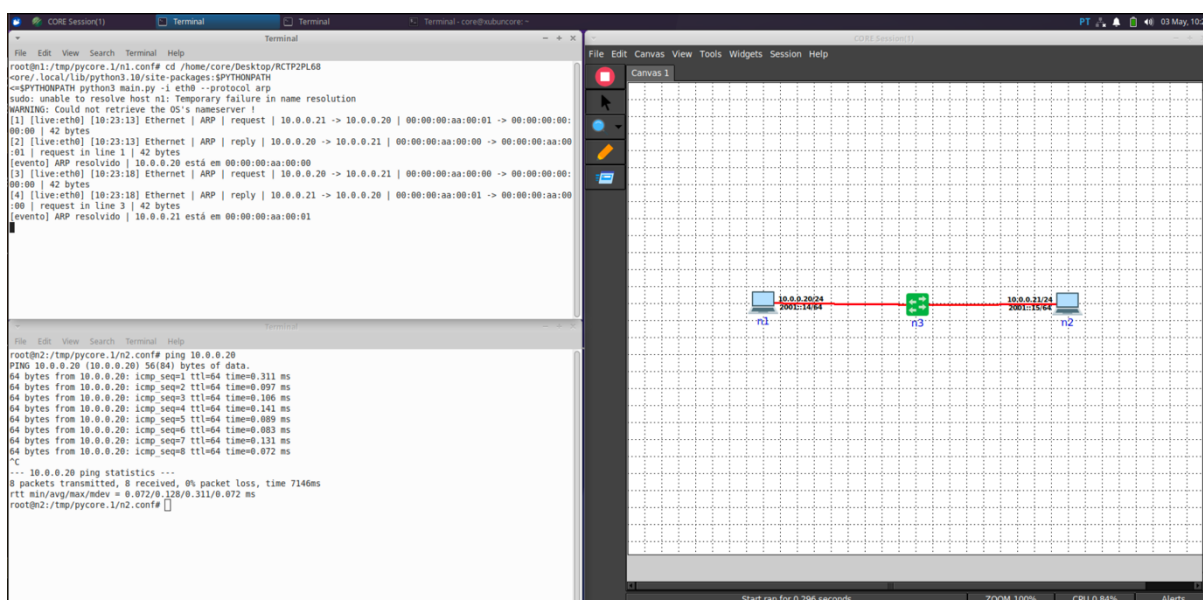


Figura 12 – Exemplo de execução do sniffer em ambiente emulado no CORE, mostrando a deteção de pedidos e respostas ARP entre nós da mesma rede virtual.

4.2. Análise na interface real

A aplicação foi também testada numa interface real do computador, com recurso a privilégios adequados, permitindo confirmar que funciona igualmente fora do contexto controlado do CORE. Foram gerados diferentes tipos de tráfego para validação: pedidos e respostas ICMP associados a `ping`, tráfego TCP e UDP de aplicações correntes, resolução ARP, padrões heurísticos de possível *traceroute* e exemplos de fragmentação IPv4.

Em interface real, a aplicação reconheceu corretamente mensagens `echo-request` e `echo-reply`, incluindo a associação `request in line ...` e o evento `ICMP reply recebido`. Em tráfego TCP e UDP foi possível observar a extração das portas, das flags TCP e de detalhes contextuais simples associados a serviços conhecidos. A aplicação detetou ainda padrões de fragmentação IPv4 com os eventos correspondentes e indicações heurísticas de possível *traceroute*, confirmando que estas funcionalidades permanecem operacionais fora do CORE.

Uma diferença relevante relativamente ao ambiente emulado é a maior variabilidade do tráfego: numa interface real surgem frequentemente protocolos e padrões não suportados nesta fase, incluindo tráfego de fundo do sistema operativo e comunicações da infraestrutura de rede. A aplicação continuou a identificar corretamente os protocolos suportados e a assinalar explicitamente os casos não tratados. Esta variabilidade permitiu também observar limitações naturais da solução: alguns tipos de tráfego surgem apenas como não suportados, e determinados padrões dependem da qualidade e completude da captura para serem reconhecidos – aspetos coerentes com o âmbito do projeto e que reforçam a distinção entre um *sniffer* académico orientado para protocolos selecionados e ferramentas generalistas como o *Wireshark*.

```
[1] [live:en0] [11:45:47] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=64 | echo-request | 84 bytes
[2] [live:en0] [11:45:47] Ethernet | IPv4 | ICMP | 8.8.8.8 -> 192.168.1.72 | ttl=116 | echo-reply | 84 bytes | request in line 1
[event] ICMP reply recebido | 8.8.8.8 respondeu a 192.168.1.72
[3] [live:en0] [11:45:48] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=64 | echo-request | 84 bytes
[4] [live:en0] [11:45:48] Ethernet | IPv4 | ICMP | 8.8.8.8 -> 192.168.1.72 | ttl=116 | echo-reply | 84 bytes | request in line 3
[event] ICMP reply recebido | 8.8.8.8 respondeu a 192.168.1.72
[5] [live:en0] [11:45:49] Ethernet | IPv4 | ICMP | 192.168.1.72 -> 8.8.8.8 | ttl=64 | echo-request | 84 bytes
[6] [live:en0] [11:45:49] Ethernet | IPv4 | ICMP | 8.8.8.8 -> 192.168.1.72 | ttl=116 | echo-reply | 84 bytes | request in line 5
[event] ICMP reply recebido | 8.8.8.8 respondeu a 192.168.1.72
```

Figura 13 – Exemplo de execução do sniffer numa interface real, mostrando a identificação de tráfego ICMP e a associação entre pedido e resposta no output da aplicação.

4.3. Testes efetuados e resultados observados

Os testes realizados, distribuídos entre o CORE e interface real conforme descrito nas secções anteriores, permitiram confirmar o correto funcionamento dos principais componentes da aplicação. Em particular, no output em modo live sobre tráfego TCP e UDP, foi possível confirmar que a reorganização da pilha protocolar manteve a informação útil e melhorou a legibilidade, com protocolos de transporte como `TCP [ACK]`, `TCP [SYN]`

ou UDP apresentados imediatamente após IPv4, seguidos dos restantes elementos relevantes do pacote.

A consulta interativa de detalhe por pacote foi igualmente testada com sucesso, permitindo visualizar informação estruturada por camadas para pacotes previamente capturados. Entre os campos observados nessa vista incluem-se os elementos de Ethernet, ARP, IPv4 e ICMP/TCP/UDP, bem como o comprimento do cabeçalho IPv4, acrescentado na fase final do desenvolvimento.

Foi ainda confirmado um ciclo completo de captura, preservação e reanálise: a aplicação escreveu corretamente a captura original para ficheiro .pcap e releu esse ficheiro em modo offline, mantendo consistência com o output produzido em modo live.

Teste	Ambiente	Resultado observado
Resolução ARP	CORE	deteção de request/reply, request in line ... e evento ARP resolvido
Ping / ICMP	Interface real	identificação de echo-request, echo-reply e evento ICMP reply recebido
Sessões TCP	Interface real	identificação de flags e deteção simplificada de estabelecimento/terminação
Fragmentação IPv4	Offline / interface real	associação entre fragmentos e evento Fragmentos IPv4 completos
Possível traceroute	Interface real	deteção heurística com base em TTL crescente
Logging estruturado	Offline / interface real	geração correta de ficheiros TXT, CSV e JSON Lines
Consulta interativa	Offline / interface real	detalhe por pacote apresentado corretamente por camadas

Tabela 5 – Síntese dos testes realizados e dos resultados observados

4.4. Obstáculos encontrados e resolução

A execução no CORE exigiu uma adaptação ao ambiente: como a biblioteca Scapy não estava imediatamente disponível dentro dos nós emulados, foi necessário utilizar o diretório do projeto a partir do sistema anfitrião virtualizado e definir explicitamente a variável PYTHONPATH com o caminho correspondente à instalação disponível, evitando assim replicar integralmente o ambiente Python em cada nó.

Ao nível da fragmentação IPv4, a aplicação evoluiu de uma apresentação simples dos campos de fragmentação para uma reconstrução virtual com agrupamento lógico dos fragmentos e verificação da cobertura dos intervalos observados. A reconstrução binária completa do *payload* não foi implementada por implicar maior complexidade sem ser indispensável para os objetivos do projeto.

Foi ainda considerada uma funcionalidade de visualização textual sob a forma de diagramas ASCII. Por exemplo, representar um *ping* como sequência de ida e volta, ou um *traceroute* com linhas de comprimento crescente refletindo o TTL e os endereços dos routers intermédios. Esta funcionalidade não foi implementada porque exigiria uma estrutura adicional de dados para reconstrução visual e um formato estável para diferentes larguras de terminal e tipos de tráfego, concluindo-se que o esforço não compensava face às prioridades centrais do projeto.

Ao nível do *output*, os ajustamentos mais relevantes foram a reorganização da pilha protocolar para apresentação seguida (por exemplo, *Ethernet | IPv4 | TCP [ACK] | ...*), a ligação explícita entre fragmentos do mesmo conjunto com referências acumuladas às linhas anteriores, e a inclusão do comprimento do cabeçalho IPv4 no detalhe interativo por pacote. Outras extensões – reconstrução profunda de fluxos TCP, *parsing* mais extenso de protocolos de aplicação, filtros por campos específicos de cabeçalho e reconstrução binária de datagramas fragmentados – foram reconhecidas como pertinentes mas deixadas de fora, privilegiando uma solução mais contida e adequada ao âmbito do trabalho.

5. Considerações

5.1. Limitações da solução

A solução desenvolvida apresenta limitações coerentes com o seu âmbito académico. A aplicação não pretende igualar ferramentas generalistas como o Wireshark (por muito que, sim, como dito logo na introdução, se inspire nelas), tendo sido construída para suportar um subconjunto controlado de protocolos e interações. Posto isto:

- A cobertura protocolar limita-se a Ethernet, ARP, IPv4, ICMP, TCP e UDP, com identificação contextual conservadora de DNS, DHCP e HTTP. Protocolos como IPv6 ou protocolos específicos de infraestrutura são assinalados como não suportados nesta fase, o que preserva a estabilidade do output mas reduz a profundidade da análise.
- O rastreador de estado é deliberadamente simples e *best effort*: o seu funcionamento depende da observação efetiva dos pacotes relevantes e da ordem em que são capturados. Se a captura começar a meio de uma interação, se faltarem pacotes ou houver ruído significativo, alguns padrões podem não ser reconhecidos ou ser reconhecidos apenas parcialmente (uma limitação particularmente relevante em redes reais).
- A fragmentação IPv4 é tratada apenas com reconstrução virtual: a aplicação agrupa logicamente fragmentos, apresenta referências entre eles e assinala quando o conjunto parece completo, mas não recompõe binariamente o *payload* original.
- Ao nível da interface, o output é textual, sem mecanismos gráficos de navegação ou visualização comparáveis aos de ferramentas especializadas. A consulta interativa por pacote melhora esta limitação, mas continua a ser uma interface essencialmente linear.

5.2. Melhorias possíveis

As linhas de evolução mais naturais da aplicação passam pelo alargamento da cobertura protocolar (nomeadamente suporte explícito a IPv6), pelo aprofundamento do *parsing* ao nível da aplicação para DNS, DHCP e HTTP, e pela reconstrução binária efetiva de datagramas fragmentados a partir dos fragmentos observados.

Ao nível da interface, podiam ser introduzidas representações textuais mais elaboradas, como os diagramas ASCII de interações considerados durante o desenvolvimento, ou uma eventual interface gráfica simples para navegação por pacotes,

eventos e fluxos. Os filtros poderiam igualmente ser aprofundados com seleção por tipos de mensagens ICMP, *flags* TCP específicas ou eventos detetados pelo rastreador.

5.2.1. Reconstrução de traceroute

A reconstrução completa da rota de um traceroute (identificação dos IPs dos routers intermédios) foi estudada e desenhada, mas não incluída na versão final.

A lógica assentava no funcionamento do protocolo: num *traceroute*, a origem envia pacotes (*probes*) com TTL progressivamente crescente. Quando o TTL expira num router, este devolve um erro `ICMP Time Exceeded` (Tipo 11). Segundo a norma, o *payload* desta mensagem de erro contém uma cópia do cabeçalho IP e os primeiros 8 bytes do pacote original.

A implementação passaria por manter um registo em memória de todos os *probes* enviados, identificados por uma chave única (ID/Sequência para ICMP, ou portas para UDP/TCP). Ao capturar um erro ICMP, o programa faria um *parsing* para extrair os dados embutidos. O cruzamento desta informação com o registo de *probes* permitiria associar a resposta à tentativa original, revelando o IP do router responsável por aquele salto (*hop*).

A funcionalidade foi suspensa devido ao tempo de desenvolvimento exigido para criar um *parsing* de pacotes encapsulados, cujos formatos variam consoante o sistema operativo. Optou-se por priorizar a estabilidade e eficiência do *sniffer*, garantindo o funcionamento robusto da deteção baseada unicamente no incremento heurístico do TTL.

6. Conclusão

De forma global, os objetivos principais do projeto foram cumpridos. A aplicação consegue capturar tráfego em modo live, ler ficheiros `.pcap` em modo offline, aplicar filtros úteis, identificar protocolos relevantes, produzir resumos textuais coerentes e gerar logs estruturados para análise posterior, integrando ainda mecanismos adicionais de valor prático e pedagógico – estatísticas finais, consulta interativa de detalhe por pacote e um rastreador de estado simples para reconhecimento de interações características.

Ao nível da identificação protocolar, o *sniffer* suporta Ethernet, ARP, IPv4, ICMP, TCP e UDP, com identificação contextual conservadora de DNS, DHCP e HTTP, o que é cobertura suficiente para demonstrar desencapsulamento por camadas, observação de campos relevantes dos cabeçalhos e interpretação básica de diferentes tipos de comunicação. Foram ainda concretizados objetivos complementares que reforçaram a qualidade final da solução: a reorganização do output para apresentar a pilha protocolar de forma mais clara, a associação explícita entre fragmentos IPv4 do mesmo conjunto, a deteção de padrões como *ping*, resolução ARP, *handshake* TCP e possível *traceroute*, e a inclusão de detalhe interativo por pacote com informação estruturada por camadas.

Os objetivos de validação foram igualmente atingidos: a aplicação foi testada em ambiente emulado no CORE e numa interface real, confirmando o seu funcionamento em contextos com características distintas. Esta comparação revelou-se especialmente útil para perceber como a mesma solução reage em cenários controlados e em tráfego efetivo, mais variado, ajudando a validar a robustez da aplicação e a compreender melhor as limitações naturais de um *sniffer* académico construído sobre um subconjunto delimitado de protocolos.

Um dos aspetos mais relevantes do desenvolvimento foi a necessidade de equilibrar ambição técnica com controlo de complexidade. Em vários momentos surgiram possibilidades de extensão, como reconstrução profunda de fluxos (detalhada no 5.2.1.), *parsing* mais extenso de protocolos de aplicação, visualizações textuais mais elaboradas... Porém, a opção final passou por privilegiar uma solução mais contida, consistente e adequadamente validada. Mais do que produzir apenas uma ferramenta funcional, o trabalho contribuiu para consolidar, de forma prática, conceitos fundamentais da unidade curricular: encapsulamento e desencapsulamento por camadas, identificação de protocolos através dos campos dos cabeçalhos, e distinção entre análise de pacotes isolados e análise relacional com manutenção de estado.